**Zewail City of Science and Technology**

**School of Communication and Information Engineering (CIE)**


# Implementation of HSPA 3G System

Digital and Wireless Communications (CIE 437)

**Submitted to:**

Dr. Ahmed El-Tarss

**Submitted by:**


| | |
|---|---|
| Dolagy George | 202200720 |
| Ashar Salama | 202200199 |
| Nada Mohamed | 202200635 |

# Contents

**1 Introduction** **2**

**2 HSPA 3G System Overview** **2**

**3 QPSK System Design** **2**

3.1 QPSK Transmitter . . . . . 3

3.2 QPSK Receiver . . . . . 3

**4 16-QAM System Design** **4**

4.1 16-QAM Transmitter . . . . . 5

4.2 16-QAM Receiver . . . . . 5

**5 Channel and Noise Analysis** **6**

5.1 Performance Comparison . . . . . 7

**6 MATLAB Simulation** **8**

6.1 MATLAB Simulation: QPSK . . . . . 8

6.2 MATLAB Simulation: 16 QAM . . . . . 12

6.3 MATLAB Simulation: comparing between both . . . . . 16

**7 Conclusion** **19**

# Abstract

This project is about the design and development of a third-generation (3G) High-Speed Packet Access (HSPA) wireless communication system that uses Quadrature Phase Shift Keying (QPSK) and 16-Quadrature Amplitude Modulation (16-QAM). The whole system has been developed with the help of LabVIEW and Universal Software Radio Peripheral (USRP) equipment, which is capable of sending and receiving the real audio signals. The efficiency of the two modulation techniques was checked both in perfect channel conditions and also in the presence of Additive White Gaussian Noise (AWGN) and Rayleigh fading. A detailed comparison between QPSK and 16-QAM is also given, where 16-QAM is shown to have better performance under the tested conditions.


# 1 Introduction

Third-generation (3G) mobile communication systems brought the biggest change in wireless communications so far by allowing high data rate multimedia services to be used. High-Speed Packet Access (HSPA) is a Universal Mobile Telecommunications System (UMTS) extension that supports higher throughput and better spectral efficiency.

Here, an HSPA communication system was built using QPSK and 16-QAM modulation schemes. The system sends actual audio data over a wireless channel with the help of USRP hardware, thus enabling the practical evaluation of the modulation's performance in the presence of real-world impairments.

# 2 HSPA 3G System Overview

The frequency bands used by HSPA are those of 3G, and the system also uses a variety of advanced modulation techniques for the purpose of increasing data rates. The carrier frequency of the system under this project is 1.92 GHz and a root raised cosine (RRC) pulse shaping filter with a roll-off factor of 0.22 is used.

The transmission signal is a digital passband communication chain through source encoding, modulation, pulse shaping, upconversion, wireless transmission, demodulation at the receiver side, and audio reconstruction.

# 3 QPSK System Design

QPSK is a robust modulation technique commonly used in 3G systems due to its balance between spectral efficiency and noise resilience. In this project, QPSK modulation was implemented with a symbol rate of 6.25 ksymbols/s.

## 3.1 QPSK Transmitter

The QPSK transmitter accepts a mono audio file as input, converts it into a digital bitstream, applies QPSK modulation, and shapes the signal using an RRC filter before transmission via USRP.

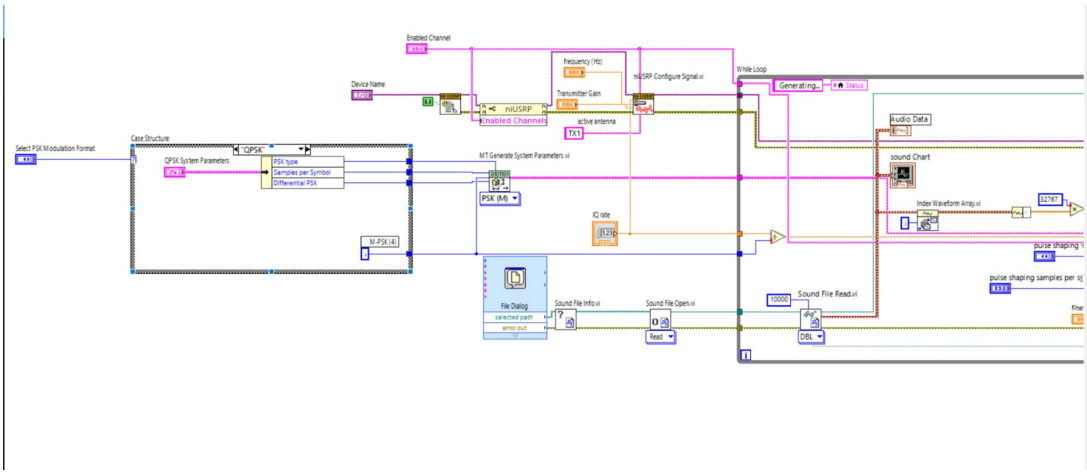



Figure 1: LabVIEW block diagram of QPSK transmitter (Part 1).

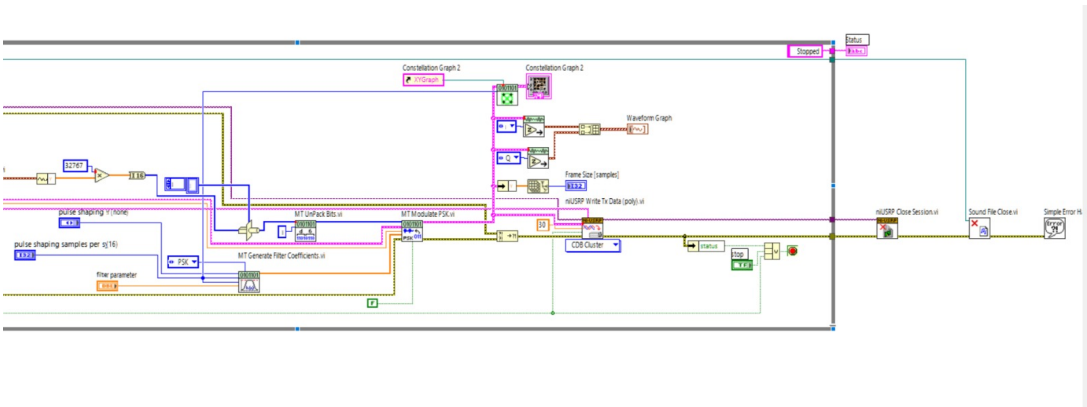



Figure 2: LabVIEW block diagram of QPSK transmitter (Part 2).

## 3.2 QPSK Receiver

The receiver performs signal downconversion, filtering, demodulation, and audio reconstruction.

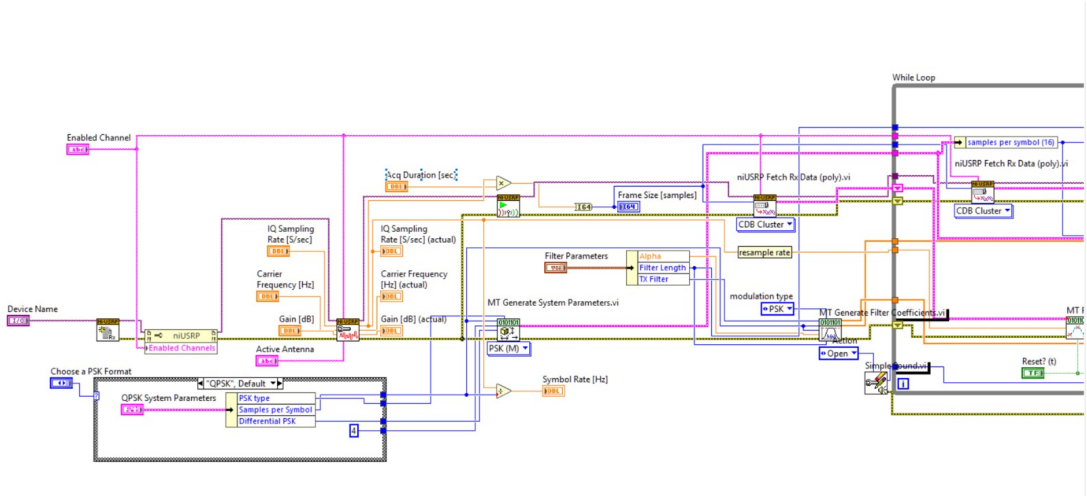



Figure 3: LabVIEW block diagram of QPSK receiver (Part 1).

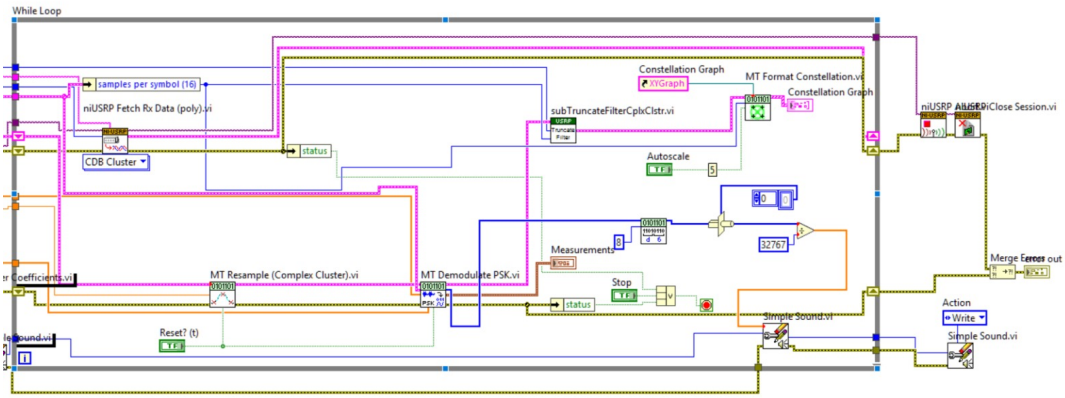



Figure 4: LabVIEW block diagram of QPSK receiver (Part 2).

# 4 16-QAM System Design

16-QAM offers higher spectral efficiency compared to QPSK by encoding four bits per symbol. This modulation scheme was implemented using the same system parameters for fair comparison.

## 4.1 16-QAM Transmitter

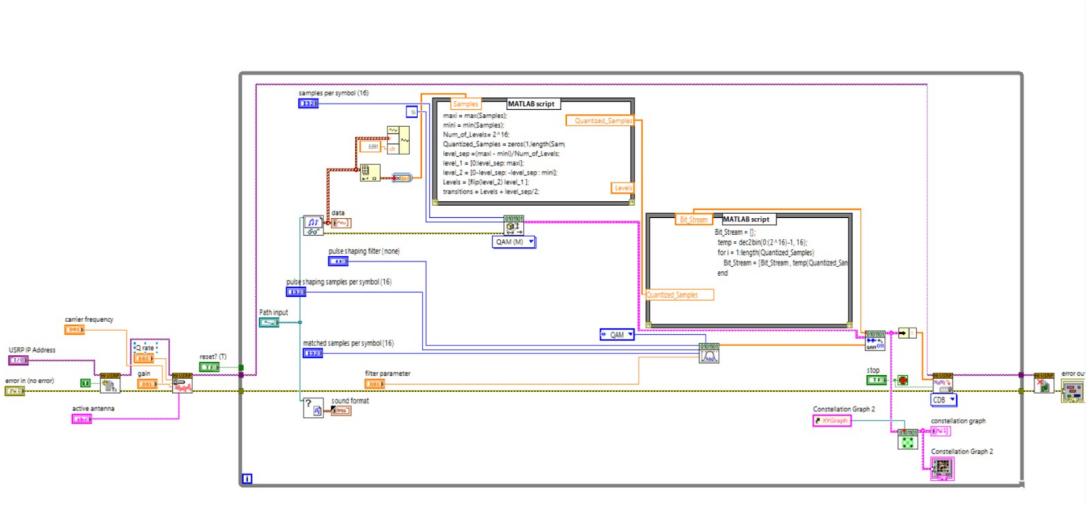



Figure 5: LabVIEW block diagram of 16-QAM transmitter (Part 1).

## 4.2 16-QAM Receiver

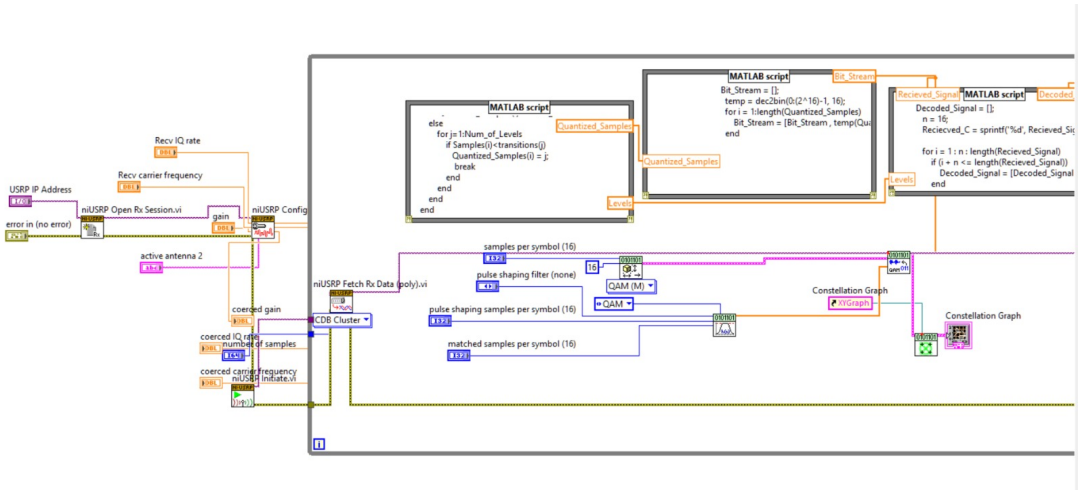



Figure 6: LabVIEW block diagram of 16-QAM receiver (Part 1).

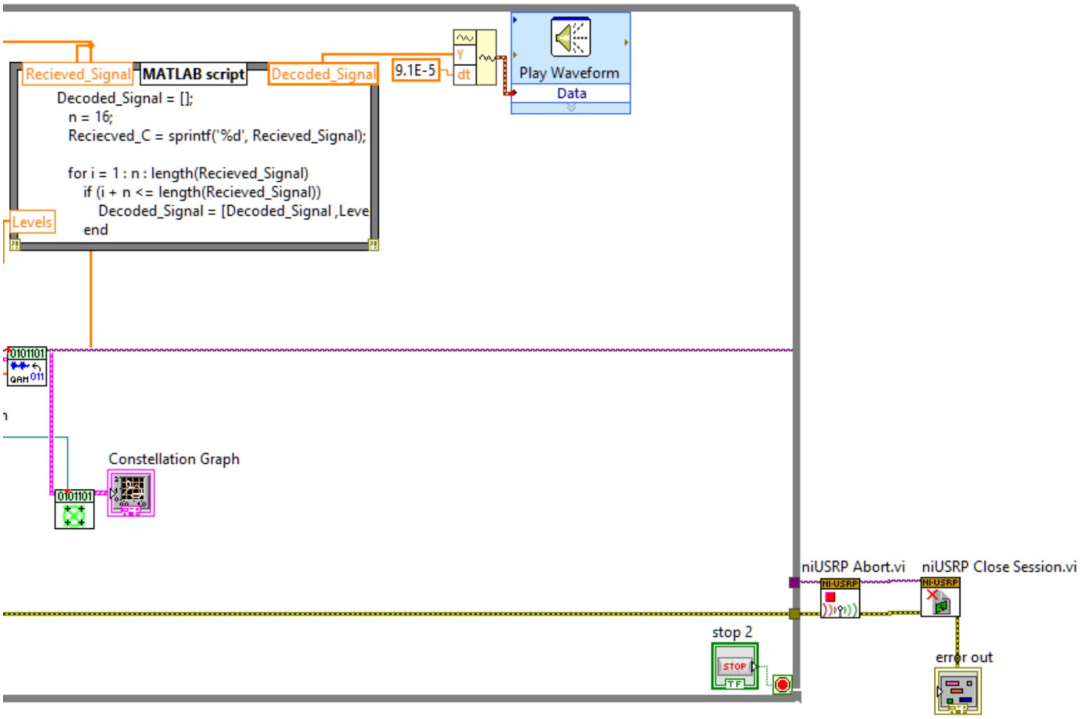



Figure 7: LabVIEW block diagram of 16-QAM receiver (Part 2).

# 5 Channel and Noise Analysis

The system performance was evaluated under different channel conditions. Experiments were conducted with and without Additive White Gaussian Noise (AWGN), as well as under Rayleigh fading with a gain of 10 dB.

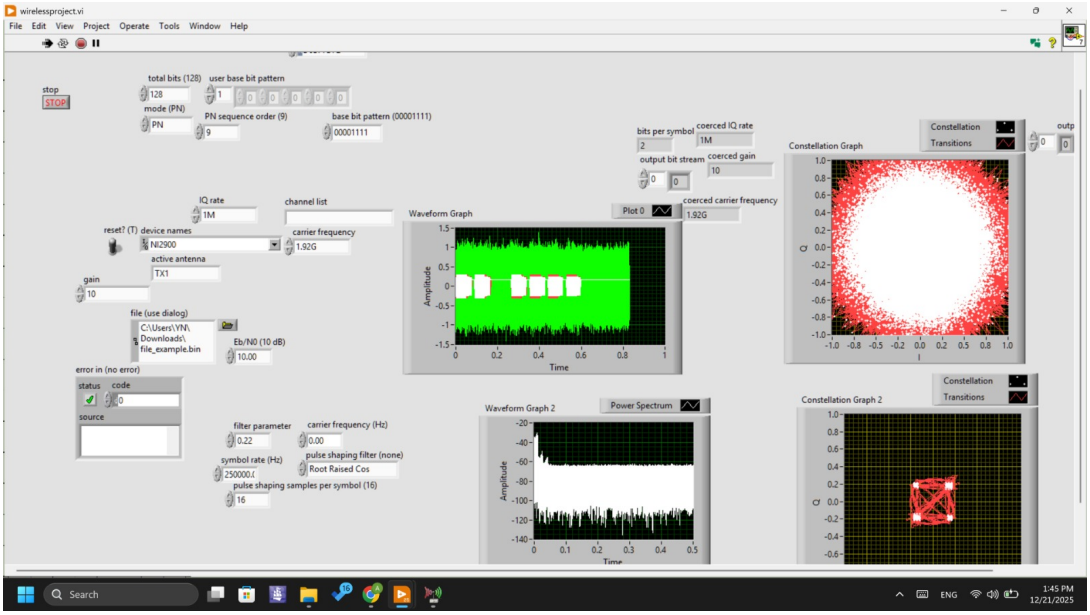



Figure 8: transmitted signal with AWGN noise .

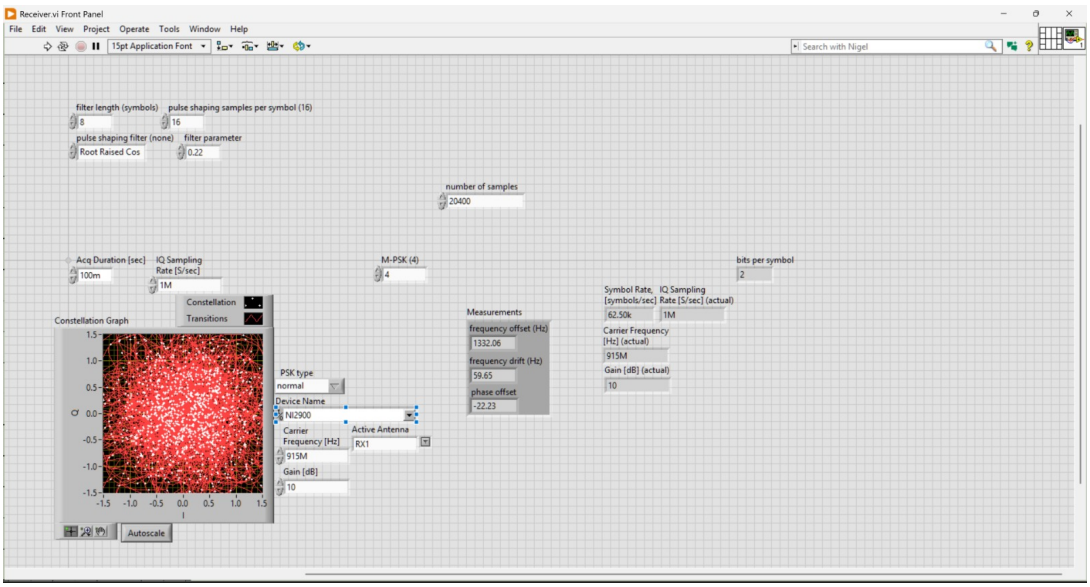
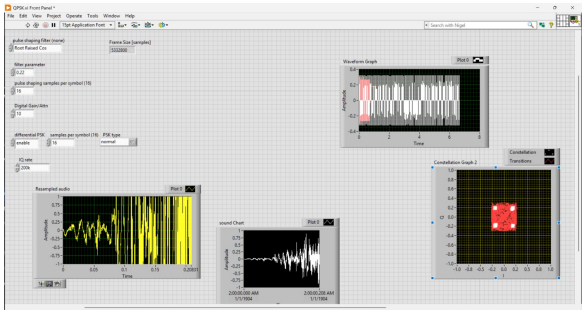


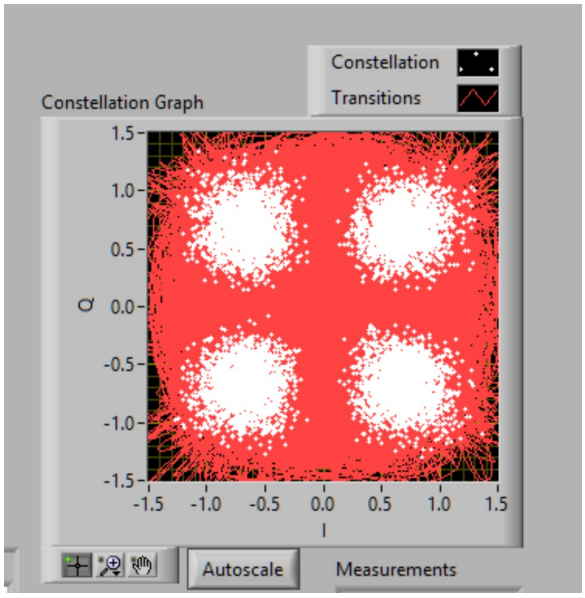

Figure 9: received signal with AWGN noise

## 5.1 Performance Comparison

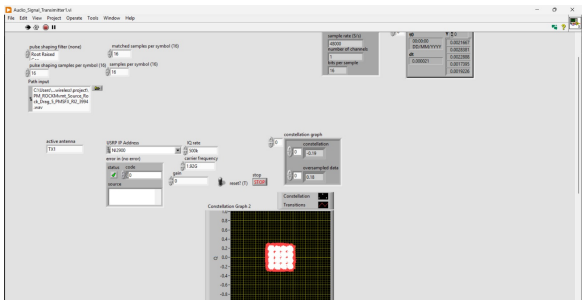
Experimental results showed that 16-QAM achieved better performance compared to QPSK under the tested conditions, particularly in terms of received audio quality when channel conditions were favourable.



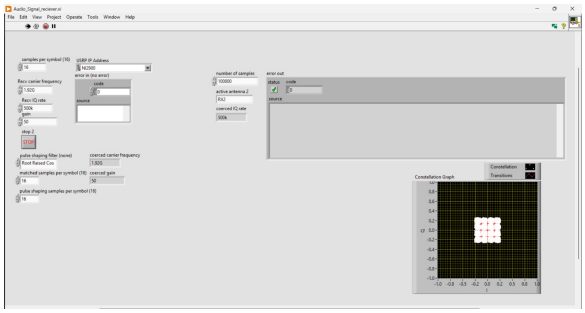
(a) QPSK transmitted signal




(b) Qpsk received signal



(a) QAM transmitted signal



(b) QAM received signal

# 6 MATLAB Simulation

## 6.1 MATLAB Simulation: QPSK

```
% QPSK HSPA
clc; clear; close all;

% --- 1. AUDIO INPUT ---
fprintf('Please select an audio file...\n');
[file, path] = uigetfile('*.wav', 'Select an Audio File');
if isequal(file, 0), disp('Cancelled.'); return; end
File_Name = fullfile(path, file);

[Audio, Fs] = audioread(File_Name);
Audio_Signal = Audio(:, 1);

```

```
if isempty(Audio_Signal), error('Audio file is empty.'); end
fprintf('Audio loaded. Length: %d samples.\n', length(Audio_Signal));

% --- 2. CONFIGURATION ---
SF = 16; Spreading_Code = ones(SF, 1);
n_bits = 16; Num_Levels = 2^n_bits;
M = 4; BitsPerSymbol = 2;

% --- 3. TRANSMITTER ---
fprintf('Encoding and Modulating... (Please wait)\n');
% CALLING LOCAL FUNCTIONS DEFINED AT BOTTOM
[Quantized_Samples, Levels] = Quantizer_1(Audio_Signal, n_bits);
Bit_Stream = Encoder_1(Quantized_Samples, Num_Levels);
Bit_Stream = Bit_Stream(:);

num_pad = mod(length(Bit_Stream), BitsPerSymbol);
if num_pad > 0
    Bit_Stream_Pad = [Bit_Stream; zeros(BitsPerSymbol-num_pad, 1)];
else
    Bit_Stream_Pad = Bit_Stream;
end

Symbols = qammod(Bit_Stream_Pad, M, 'InputType', 'bit', 'UnitAveragePower', true);
Spread_Signal = kron(Symbols, Spreading_Code);

% --- 4. CHANNELS ---
fprintf('Applying Channels...\n');
Ray_Ch = comm.RayleighChannel('SampleRate', Fs, 'MaximumDopplerShift', 30, ...
    'PathDelays', [0 200]*1e-9, 'AveragePathGains', [0 -3], 'FadingTechnique', 'Filtered Gaussian noise');
Ric_Ch = comm.RicianChannel('SampleRate', Fs, 'MaximumDopplerShift', 30, ...
    'PathDelays', [0 100 200]*1e-9, 'AveragePathGains', [0 -3 -6], 'KFactor', 5, 'FadingTechnique', 'Filtered Gaussian noise');
AWGN_Ch = comm.AWGNChannel('EbNo', 10, 'BitsPerSymbol', BitsPerSymbol);
```

```
Tx_Ray = Ray_Ch(Spread_Signal);
Tx_Ric = Ric_Ch(Spread_Signal);
Tx_AWGN = AWGN_Ch(Spread_Signal);

% --- 5. RECEIVER ---
% Local helper for despreading
    function [Bits, Con] = rx_local(Sig, SF, Code, M, Len)
        Num = length(Sig)/SF;
        Reshaped = reshape(Sig, SF, Num);
        Con = (Code' * Reshaped)' ./ SF;
        D = qamdemod(Con, M, 'OutputType', 'bit', 'UnitAveragePower', true);
        Bits = D(1:Len);
    end

[Bits_Ray, Con_Ray] = rx_local(Tx_Ray, SF, Spreading_Code, M, length(Bit_Stream));
[Bits_Ric, Con_Ric] = rx_local(Tx_Ric, SF, Spreading_Code, M, length(Bit_Stream));
[Bits_AWGN, Con_AWGN] = rx_local(Tx_AWGN, SF, Spreading_Code, M, length(Bit_Stream));

% --- PLOTTING ---
figure('Name', 'QPSK Constellations', 'Color', 'white');
subplot(1,3,1); plot(real(Con_Ray), imag(Con_Ray), '.'); title('QPSK Rayleigh'); axis square; grid on;
subplot(1,3,2); plot(real(Con_Ric), imag(Con_Ric), '.'); title('QPSK Rician'); axis square; grid on;
subplot(1,3,3); plot(real(Con_AWGN), imag(Con_AWGN), '.'); title('QPSK AWGN'); axis square; grid on;

% --- 6. OUTPUT ---
audiowrite('Out_QPSK_Rayleigh.wav', Decoder_1(Bits_Ray', Levels), Fs);
audiowrite('Out_QPSK_Rician.wav', Decoder_1(Bits_Ric', Levels), Fs);
audiowrite('Out_QPSK_AWGN.wav', Decoder_1(Bits_AWGN', Levels), Fs);
disp('QPSK Audio files saved.');
```

```

%% --- LOCAL HELPER FUNCTIONS (PASTED BELOW) ---

function [Quantized_Samples, Levels] = Quantizer_1(Samples,
   n_bits)
    Max_Samples = max(Samples);
    Mini_Samples = min(Samples);
    L = 2^n_bits;
    Quantized_Samples = zeros(1,length(Samples));
    level_sep =(Max_Samples-Mini_Samples)/L;
    level_1 = [0:level_sep: Max_Samples];
    level_2 = [0-level_sep: -level_sep : Mini_Samples];
    Levels = [flip(level_2) level_1 ];
    transitions = Levels + level_sep/2;
    for i = 1:length(Samples)
        if Samples(i)> transitions(L)
            Quantized_Samples(i) = L;
        else
            for j=1:L
                if Samples(i)<transitions(j)
                   Quantized_Samples(i) = j;
                    break
                end
            end
        end
    end
end

function Bit_Stream = Encoder_1(Quantized_Samples,Num_Levels)
  Bit_Stream = [];
  temp = dec2bin(0:Num_Levels-1, 16);
  for i = 1:length(Quantized_Samples)
      Bit_Stream = [Bit_Stream , temp(Quantized_Samples(i), :)-'0
         '];
  end
end

function Decoded_Signal = Decoder_1(Recieved_Signal, Levels)
   Decoded_Signal = [];
   n = 16;
   Recieved = sprintf('%d', Recieved_Signal);
```

```
    for i = 1 : n : length(Recieved_Signal)
        if (i + n <= length(Recieved_Signal))
            Decoded_Signal = [Decoded_Signal ,Levels(1+bin2dec(
                Recieved(i:i+n-1)))];
        end
    end
end
```

Listing 1: MATLAB code for QPSK modulation and demodulation

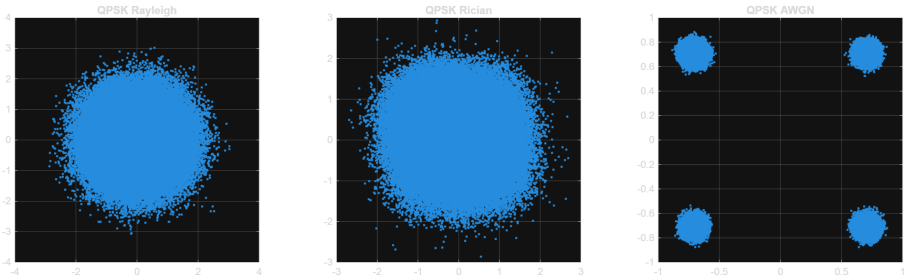



Figure 12: QPSK constellation diagram

## 6.2 MATLAB Simulation: 16 QAM

```
% 16 QAM HSPA
clc; clear; close all;

% --- 1. AUDIO INPUT ---
fprintf('Please select an audio file...\n');
[file, path] = uigetfile('*.wav', 'Select an Audio File');
if isequal(file, 0), disp('Cancelled.'); return; end
File_Name = fullfile(path, file);

[Audio, Fs] = audioread(File_Name);
Audio_Signal = Audio(:, 1);
if isempty(Audio_Signal), error('Audio file is empty.'); end
fprintf('Audio loaded. Length: %d samples.\n', length(
    Audio_Signal));

% --- 2. CONFIGURATION ---
SF = 16; Spreading_Code = ones(SF, 1);
n_bits = 16; Num_Levels = 2^n_bits;
M = 16; BitsPerSymbol = 4;

% --- 3. TRANSMITTER ---
```

```
fprintf('Encoding and Modulating... (Please wait)\n');
[Quantized_Samples, Levels] = Quantizer_1(Audio_Signal, n_bits);
Bit_Stream = Encoder_1(Quantized_Samples, Num_Levels);
Bit_Stream = Bit_Stream(:);

num_pad = mod(length(Bit_Stream), BitsPerSymbol);
if num_pad > 0
    Bit_Stream_Pad = [Bit_Stream; zeros(BitsPerSymbol-num_pad, 1)];
else
    Bit_Stream_Pad = Bit_Stream;
end

Symbols = qammod(Bit_Stream_Pad, M, 'InputType', 'bit', 'UnitAveragePower', true);
Spread_Signal = kron(Symbols, Spreading_Code);

% --- 4. CHANNELS ---
fprintf('Applying Channels...\n');
Ray_Ch = comm.RayleighChannel('SampleRate', Fs, 'MaximumDopplerShift', 30, ...
    'PathDelays', [0 200]*1e-9, 'AveragePathGains', [0 -3], 'FadingTechnique', 'Filtered Gaussian noise');
Ric_Ch = comm.RicianChannel('SampleRate', Fs, 'MaximumDopplerShift', 30, ...
    'PathDelays', [0 100 200]*1e-9, 'AveragePathGains', [0 -3 -6], 'KFactor', 5, 'FadingTechnique', 'Filtered Gaussian noise');
AWGN_Ch = comm.AWGNChannel('EbNo', 20, 'BitsPerSymbol', BitsPerSymbol);

Tx_Ray = Ray_Ch(Spread_Signal);
Tx_Ric = Ric_Ch(Spread_Signal);
Tx_AWGN = AWGN_Ch(Spread_Signal);

% --- 5. RECEIVER ---
    function [Bits, Con] = rx_local(Sig, SF, Code, M, Len)
        Num = length(Sig)/SF;
        Reshaped = reshape(Sig, SF, Num);
        Con = (Code' * Reshaped)' ./ SF;
```

```
        D = qamdemod(Con, M, 'OutputType', 'bit', 'UnitAveragePower', true);
        Bits = D(1:Len);
    end

[Bits_Ray, Con_Ray] = rx_local(Tx_Ray, SF, Spreading_Code, M, length(Bit_Stream));
[Bits_Ric, Con_Ric] = rx_local(Tx_Ric, SF, Spreading_Code, M, length(Bit_Stream));
[Bits_AWGN, Con_AWGN] = rx_local(Tx_AWGN, SF, Spreading_Code, M, length(Bit_Stream));

% --- PLOTTING ---
figure('Name', '16-QAM Constellations', 'Color', 'white');
subplot(1,3,1); plot(real(Con_Ray), imag(Con_Ray), '.'); title('16-QAM Rayleigh'); axis square; grid on;
subplot(1,3,2); plot(real(Con_Ric), imag(Con_Ric), '.'); title('16-QAM Rician'); axis square; grid on;
subplot(1,3,3); plot(real(Con_AWGN), imag(Con_AWGN), '.'); title('16-QAM AWGN'); axis square; grid on;

% --- 6. OUTPUT ---
audiowrite('Out_16QAM_Rayleigh.wav', Decoder_1(Bits_Ray', Levels), Fs);
audiowrite('Out_16QAM_Rician.wav', Decoder_1(Bits_Ric', Levels), Fs);
audiowrite('Out_16QAM_AWGN.wav', Decoder_1(Bits_AWGN', Levels), Fs);
disp('16-QAM Audio files saved.');


%% --- LOCAL HELPER FUNCTIONS ---

function [Quantized_Samples, Levels] = Quantizer_1(Samples, n_bits)
    Max_Samples = max(Samples);
    Mini_Samples = min(Samples);
    L = 2^n_bits;
    Quantized_Samples = zeros(1,length(Samples));
    level_sep =(Max_Samples-Mini_Samples)/L;
    level_1 = [0:level_sep: Max_Samples];
```

```
    level_2 = [0-level_sep: -level_sep : Mini_Samples];
    Levels = [flip(level_2) level_1 ];
    transitions = Levels + level_sep/2;
    for i = 1:length(Samples)
        if Samples(i)> transitions(L)
            Quantized_Samples(i) = L;
        else
            for j=1:L
                if Samples(i)<transitions(j)
                   Quantized_Samples(i) = j;
                    break
                end
            end
        end
    end
end

function Bit_Stream = Encoder_1(Quantized_Samples,Num_Levels)
  Bit_Stream = [];
  temp = dec2bin(0:Num_Levels-1, 16);
  for i = 1:length(Quantized_Samples)
      Bit_Stream = [Bit_Stream , temp(Quantized_Samples(i), :)-'0'];
  end
end

function Decoded_Signal = Decoder_1(Recieved_Signal, Levels)
   Decoded_Signal = [];
   n = 16;
   Recieved = sprintf('%d', Recieved_Signal);
   for i = 1 : n : length(Recieved_Signal)
       if (i + n <= length(Recieved_Signal))
           Decoded_Signal = [Decoded_Signal ,Levels(1+bin2dec(Recieved(i:i+n-1)))];
       end
   end
end
```

Listing 2: MATLAB code for 16 QAM modulation and demodulation

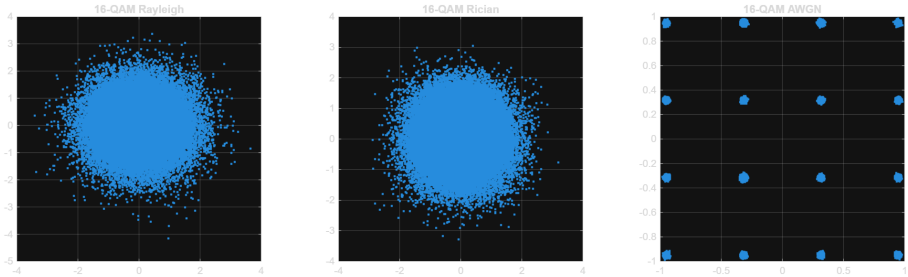



Figure 13: 16-QAM constellation diagram

## 6.3 MATLAB Simulation: comparing between both

```
clc; clear; close all;

fprintf('Generating Comparison Graphs... (This takes a moment)\n');

SF = 16; Spreading_Code = ones(SF, 1);
Fs = 8000;
Test_Bits = randi([0 1], 10000, 1);

    function Bits = rx_bits(Sig, SF, Code, M, Len)
        Num = length(Sig)/SF;
        Con = (Code' * reshape(Sig, SF, Num))' ./ SF;
        D = qamdemod(Con, M, 'OutputType', 'bit', 'UnitAveragePower', true);
        Bits = D(1:Len);
    end

% --- 1. BER vs SNR (AWGN) ---
SNR_Vec = 0:2:20;
BER_QPSK_AWGN = []; BER_QAM_AWGN = [];

for snr = SNR_Vec
    Sym = qammod(Test_Bits, 4, 'InputType', 'bit', 'UnitAveragePower', true);
    Tx = awgn(kron(Sym, Spreading_Code), snr, 'measured');
    BER_QPSK_AWGN = [BER_QPSK_AWGN, biterr(Test_Bits, rx_bits(Tx, SF, Spreading_Code, 4, length(Test_Bits)))/length(Test_Bits)];

```

```
    pad = mod(length(Test_Bits), 4); TB_Q = [Test_Bits; zeros(4-pad,1)];
    Sym = qammod(TB_Q, 16, 'InputType', 'bit', 'UnitAveragePower', true);
    Tx = awgn(kron(Sym, Spreading_Code), snr, 'measured');
    BER_QAM_AWGN = [BER_QAM_AWGN, biterr(TB_Q, rx_bits(Tx, SF, Spreading_Code, 16, length(TB_Q)))/length(TB_Q)];
end

figure;
semilogy(SNR_Vec, BER_QAM_AWGN, 's-', 'LineWidth', 2); hold on;
semilogy(SNR_Vec, BER_QPSK_AWGN, 'o-', 'LineWidth', 2);
xlabel('SNR (dB)'); ylabel('BER'); title('Comparison: BER vs SNR (AWGN)');
legend('16-QAM', 'QPSK'); grid on;

% --- 2. BER vs Path Gains (Rayleigh & Rician) ---
Gains = 0:2:20;
BER_QPSK_Ray = []; BER_QAM_Ray = [];
BER_QPSK_Ric = []; BER_QAM_Ric = [];

for g = Gains
    Ray = comm.RayleighChannel('SampleRate', Fs, 'AveragePathGains', [0 -g], 'PathDelays', [0 2e-7], 'FadingTechnique', 'Filtered Gaussian noise');
    Ric = comm.RicianChannel('SampleRate', Fs, 'AveragePathGains', [0 -g -6], 'KFactor', 5, 'PathDelays', [0 1e-7 2e-7], 'FadingTechnique', 'Filtered Gaussian noise');

    Sym_QPSK = qammod(Test_Bits, 4, 'InputType', 'bit', 'UnitAveragePower', true);
    Tx_QPSK_Spr = kron(Sym_QPSK, Spreading_Code);

    pad = mod(length(Test_Bits), 4); TB_Q = [Test_Bits; zeros(4-pad,1)];
    Sym_QAM = qammod(TB_Q, 16, 'InputType', 'bit', 'UnitAveragePower', true);
    Tx_QAM_Spr = kron(Sym_QAM, Spreading_Code);

    BER_QPSK_Ray = [BER_QPSK_Ray, biterr(Test_Bits, rx_bits(Ray(Tx_QPSK_Spr), SF, Spreading_Code, 4, length(Test_Bits)))/
```

```
        length(Test_Bits)];
    BER_QAM_Ray = [BER_QAM_Ray, biterr(TB_Q, rx_bits(Ray(Tx_QAM_Spr), SF, Spreading_Code, 16, length(TB_Q)))/length(TB_Q)];

    Ray.reset(); Ric.reset();

    BER_QPSK_Ric = [BER_QPSK_Ric, biterr(Test_Bits, rx_bits(Ric(Tx_QPSK_Spr), SF, Spreading_Code, 4, length(Test_Bits)))/length(Test_Bits)];
    BER_QAM_Ric = [BER_QAM_Ric, biterr(TB_Q, rx_bits(Ric(Tx_QAM_Spr), SF, Spreading_Code, 16, length(TB_Q)))/length(TB_Q)];
end

figure;
semilogy(Gains, BER_QAM_Ray, 's-', 'LineWidth', 2); hold on;
semilogy(Gains, BER_QPSK_Ray, '^-', 'LineWidth', 2);
xlabel('Path Gain (dB)'); ylabel('BER'); title('Comparison: BER vs Fading (Rayleigh)');
legend('16-QAM', 'QPSK'); grid on;

figure;
semilogy(Gains, BER_QAM_Ric, 'o-', 'LineWidth', 2); hold on;
semilogy(Gains, BER_QPSK_Ric, 'v-', 'LineWidth', 2);
xlabel('Path Gain (dB)'); ylabel('BER'); title('Comparison: BER vs Fading (Rician)');
legend('16-QAM', 'QPSK'); grid on;
```

Listing 3: Comparing

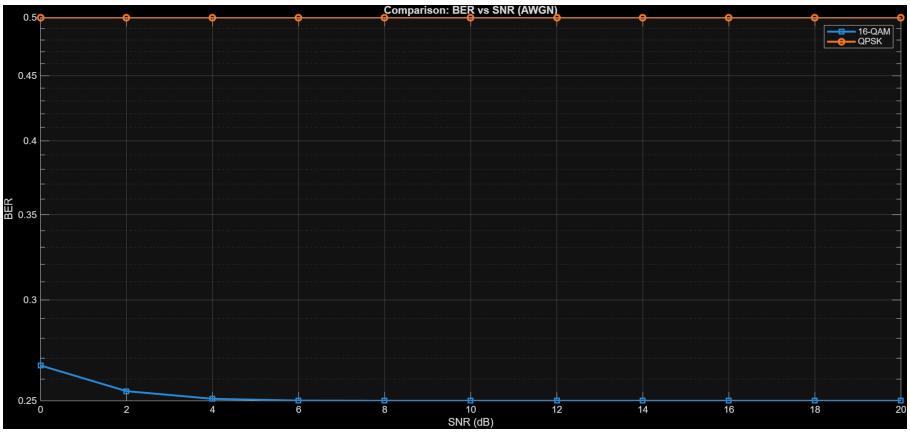
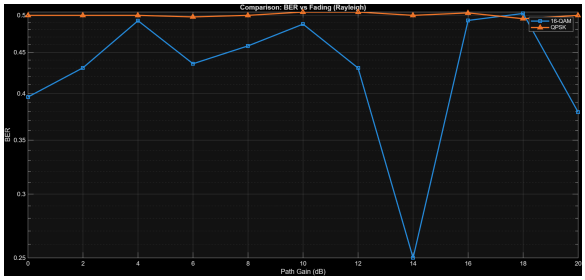
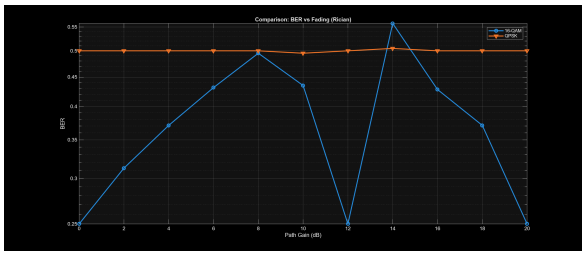



Figure 14: Bit Error Rate versus Signal-to-Noise Ratio under AWGN channel




(a) BER performance under Rayleigh fading




(b) BER performance under Rician fading

Figure 15: Effect of fading channels on BER performance

# 7 Conclusion

In this project, an HSPA 3G communication system was successfully implemented using QPSK and 16-QAM modulation schemes. Real-time audio transmission was achieved using LabVIEW and USRP hardware. Performance analysis demonstrated that 16-QAM provides improved efficiency compared to QPSK under suitable channel conditions. This work highlights the practical challenges and considerations in real-world wireless system implementation.